

مستندات فنی و زیرساخت — BotLab

Oct 6, 2026 • @Claudeuser

خلاصه

ایجنتی است که از توضیح فارسیِ صاحب کسب و کار، بات آمادهٔ بله یا تلگرام می‌سازد، آن را در BotLab محیط آزمایشی می‌آزماید، منتشر می‌کند و تغییرات بعدی را با نسخهٔ جدید و آزمون رگرسیون اعمال اعتبارسنجی شده (BotSpec) می‌کند. ایجنت هیچ کد اجرایی تولید نمی‌کند؛ خروجی او یک «مشخصات بات» است که یک موتور قطعی اجرا می‌کند.

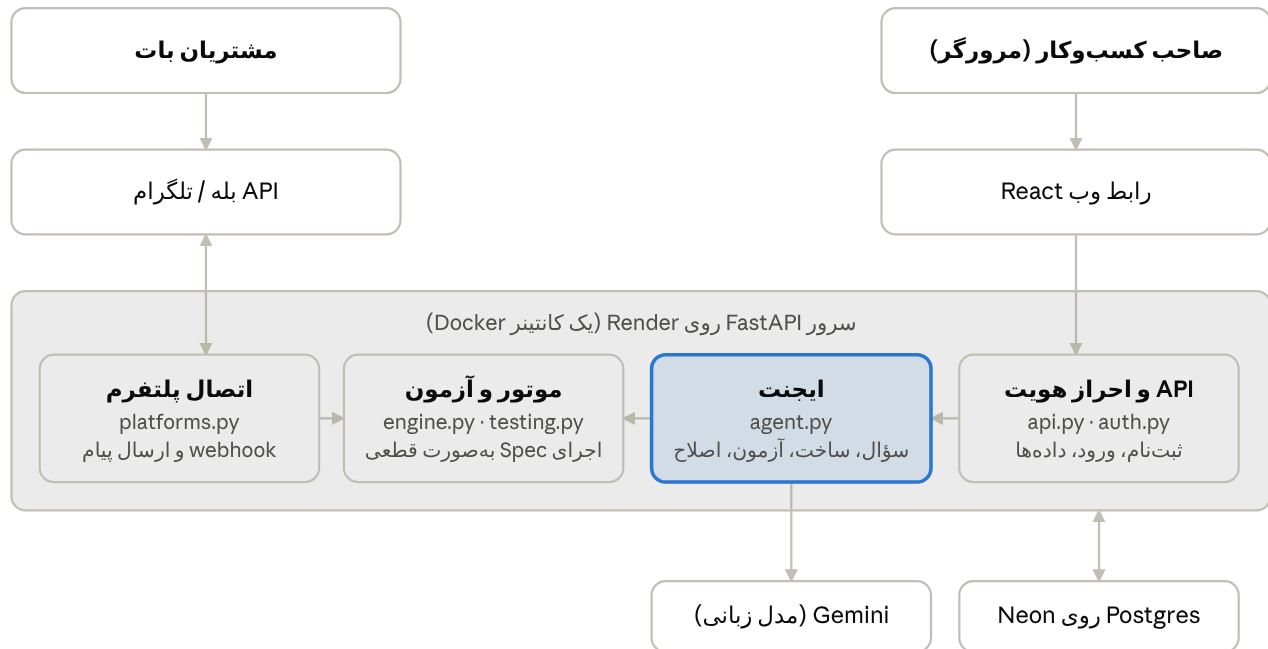
مورد	مقدار
مسئلهٔ مسابقه	سازنده و نگه‌دارندهٔ یک نوع بات تلگرام یا بله
نسخهٔ آنلاین	botlab-cau2.onrender.com
سورس کد	github.com/zhaleh197/BotLab
قالب‌های نسخهٔ اول	رزرو کارگاه (ظرفیت، ثبت‌نام، لغو، فهرست انتظار) و ثبت سفارش از منو (سبد، گزینه، حداقل سفارش، ارسال، ساعت کاری)
پلتفرم‌ها	(سازگار API) بله و تلگرام

جریان اصلی کاربر: ثبت‌نام و ورود ← توضیح نیاز ← پاسخ به سؤال‌های ایجنت ← ساخت و آزمون خودکار ← امتحان در شبیه‌ساز ← انتشار با توکن بات ← درخواست تغییر ← نسخهٔ جدید و آزمون دوباره.

معماری سیستم

ایجنت و اجرای بات‌ها را با هم ارائه می‌دهد، API، کل محصول یک سرویس واحد است که رابط وب

ایجنت فقط Spec می‌سازد؛ یک موتور، هم آزمون و هم بات واقعی را اجرا می‌کند



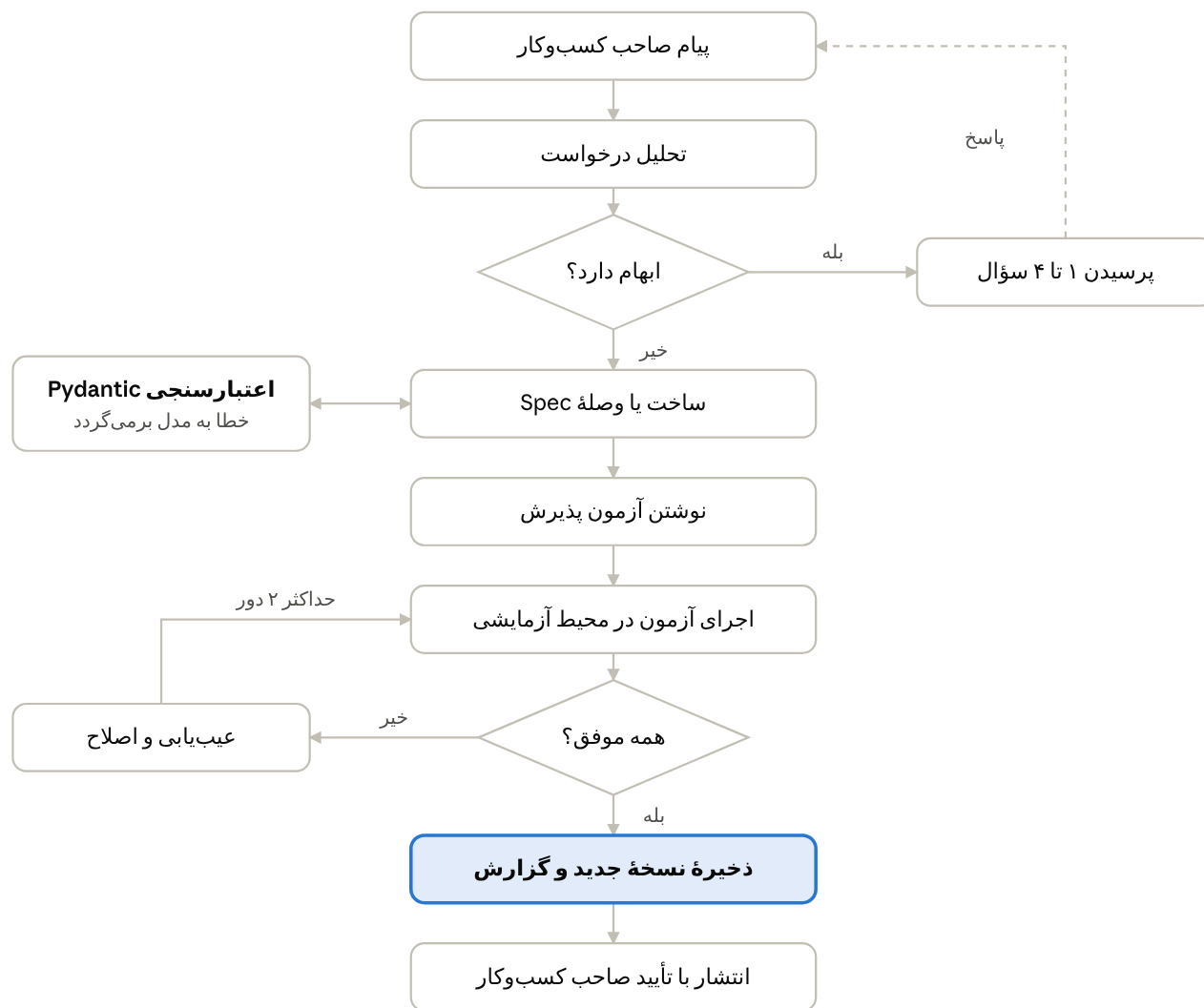
یک سرویس، ۴ ماژول، ۳ سرویس بیرونی، BotLab معماری

صاحب کسب‌وکار از مسیر سمت راست با ايجنت کار می‌کند و مشتریان از مسیر سمت چپ با بات؛ هر دو مسیر به همان موتور می‌رسند، و مدل زبانی فقط از مسیر ايجنت فراخوانی می‌شود.

جریان اجرای ايجنت

هر پیام صاحب کسب‌وکار یک «نوبت» ايجنت را در پس‌زمینه اجرا می‌کند؛ هر گام همان لحظه در پنل گفتگو نمایش داده می‌شود.

هیچ نسخه‌ای بدون اجرای آزمون‌ها ساخته نمی‌شود



جریان یک نوبت ایجنت ۲۰ تصمیم، ۲ حلقه اصلاح

نمایش diff فعلی وصله می‌شود، تفاوت دو نسخه به صورت Spec: درخواست تغییر از همین مسیر می‌گذرد داده می‌شود و آزمون‌های نسخه قبل همراه آزمون‌های جدید دوباره اجرا می‌شوند. سؤال‌ها حداکثر دو دور پرسیده می‌شوند؛ پس از آن ایجنت با پیش‌فرض‌های اعلام‌شده جلو می‌رود. اگر مدل زبانی در میانه کار در دسترس نباشد، نسخه با آزمون‌های خودکار و رگرسیون ذخیره می‌شود.

مدل‌ها و ابزارها

مدل زبانی از طریق سرویس‌دهنده فراخوانی می‌شود و هیچ مدلی محلی اجرا نمی‌شود. کلاینت هم استفاده کرد OpenRouter یا Groq است؛ با تغییر دو متغیر محیطی می‌توان از OpenAI-compatible استفاده کرد.

موضوع	انتخاب	دلیل
مدل اصلی	Google Gemini <code>gemin-3.5-flash</code>	سطح رایگان، فارسی خوب، پاسخ حدود ۱۰ ثانیه
مدل‌های جایگزین	<code>gemin-flash-latest</code> , <code>gemin-3.5-flash-lite</code>	در خطای 503 (شلوغی) یا 429 (سهمیه) خودکار جایگزین می‌شوند
قالب خروجی	به علاوه اعتبارسنجی JSON mode Pydantic	هر خروجی نامعتبر با متن خطا به مدل برمی‌گردد
ارکستراسیون ایجنت	(<code>agent.py</code>) گراف گام‌ها در پایتون	کنترل کامل و قابل پیش‌بینی، بدون ایجنت‌ساز
بک‌اند	FastAPI, SQLAlchemy 2, Pydantic 2, httpx, bcrypt, PyJWT	سبک و استاندارد
فرانت	React 18, Vite 6, Tailwind 4, فونت وزیرمتن	رابط راست‌به‌چپ و سریع

نویسندهٔ آزمون و Spec هر گام ایجنت یک فراخوانی جداگانه با پرامپت تخصصی است؛ تحلیلگر، سازندهٔ عیب‌یاب، یک ساخت کامل معمولاً ۴ تا ۶ فراخوانی مدل می‌برد.

و موتور اجرا BotSpec

تولید می‌کند. نتیجه: هر (BotSpec) مهم‌ترین تصمیم فنی این است که ایجنت به‌جای کد، یک توصیف اعلانی بات امن است (کد دلخواه اجرا نمی‌شود)، قابل آزمون است (همان موتور در تست و تولید اجرا می‌شود) و (است JSON از diff تفاوت دو نسخه یک) نسخه‌پذیر است.

(: `spec.py` در Pydantic تعریف شده با) **BotSpec ساختار**

بخش	فیلدها
مشترک	<code>template</code> , <code>business_name</code> , <code>welcome_message</code> , <code>currency</code> , <code>support_contact</code>
(workshop) کارگاه	فهرست جلسه‌ها (شناسه، عنوان، زمان، ظرفیت، قیمت) <code>allow_cancel</code> , <code>max_per_user</code> , <code>waitlist</code> (فعال)، <code>collect_fields</code> (سقف)

بخش	فیلدها
سفارش (order)	آیتم‌ها (نام، قیمت، دسته، موجودی، گزینه‌ها)، قواعد (حداقل سفارش، سقف تعداد، ارسال و آستانه ارسال رایگان، collect_fields، ساعت کاری)

اعتبارسنجی شامل یکتایی شناسه‌ها و عنوان‌ها، بازه اعداد و سازگاری بخش‌ها با قالب است. شناسه‌ها در نسخه‌های بعد ثابت می‌مانند تا داده‌های واقعی (ثبت‌نام‌ها) با انتشار نسخه جدید حفظ شوند.

یک پیام می‌گیرد و (کاربر، متن) `handle`: یک ماشین حالت قطعی است (`engine.py`) **موتور اجرا** پیام‌های خروجی را برمی‌گرداند، از جمله پیام به کاربران دیگر (مثلاً اطلاع به نفر اول فهرست انتظار). همه نگهداری می‌شود JSON حالت در یک شیء

- **کارگاه:** فهرست کارگاه‌ها با ظرفیت باقی‌مانده، ثبت‌نام با دریافت فیلدها و تأیید، کد پیگیری، «ثبت‌نام‌های من»، لغو، فهرست انتظار با ارتقای خودکار و اطلاع‌رسانی
- **سفارش:** منوی دسته‌بندی‌شده، انتخاب گزینه، سبد با افزودن و کم کردن، بررسی قواعد در زمان نهایی کردن، محاسبه هزینه ارسال، شماره سفارش
- **مشترک:** اعتبارسنجی شماره موبایل و نام، تبدیل ارقام فارسی، انصراف از هر مرحله و حذف خودکار. آیتم‌هایی که در نسخه جدید ناموجود شده‌اند

آزمون در محیط آزمایشی

هر نسخه پیش از ذخیره با دو دسته آزمون سنجیده می‌شود. هر آزمون یک گفتگوی اسکریپتی است که در (`testing.py`) حافظه و با داده خالی اجرا می‌شود

دسته	سازنده	چه می‌سنجد
آزمون خودکار	Spec کد، از روی	شروع، ثبت‌نام یا سفارش کامل، پر شدن ظرفیت و فهرست انتظار، حداقل سفارش، ساعت کاری
آزمون پذیرش	ایجت	قواعد خاص همین صاحب کسب‌وکار و آخرین تغییر

هر گام آزمون می‌گوید کدام کاربر چه می‌فرستد، پاسخ باید شامل چه باشد و نباید شامل چه باشد، و چه کاربران دیگری باید پیام بگیرند. مقایسه پس از یکسان‌سازی ارقام فارسی، ی/ک عربی و جداکننده هزارگان انجام می‌شود.

- **رگرسیون:** آزمون‌های پذیرش هر نسخه به نسخه بعد منتقل و دوباره اجرا می‌شوند. اگر تغییر عمده صاحب کسب‌وکار آزمونی را بی‌اعتبار کند، عیب‌یاب آن را به‌روز یا حذف می‌کند و دلیلش را ثبت می‌کند
- است یا از آزمون؛ **اصلاح خودکار:** برای هر شکست، عیب‌یاب تشخیص می‌دهد اشکال از قابل رفع‌اند، نه با تضعیف آزمون Spec حداکثر دو دور. آزمون‌های خودکار فقط با اصلاح

- **شبیه ساز:** صاحب کسب و کار یا داور بدون توکن بات، با سه کاربر آزمایشی با بات گفتگو می کند. داده. شبیه ساز از داده واقعی جداست.

بات کافه ۷ از ۷ و پس از تغییر ۹ از ۹؛ بات کارگاه ۵ از ۵، هر دو پس از یک Gemini نتیجه اجرای واقعی با دور اصلاح خودکار آزمون نادرست.

انتشار روی بله و تلگرام

انتشار یک اقدام صریح صاحب کسب و کار است؛ ایجنت نسخه آزموده را آماده می کند و انسان تصمیم تلگرام سازگار است، پس یک کلاینت برای هر دو کافی است API با (`tapi.bale.ai`) بله API. می گیرد (`platforms.py`).

1. تأیید و نام کاربری بات ذخیره می شود `getMe` توکن با.
 2. با آدرس `setWebhook`، (خودکار Render روی) اگر آدرس عمومی سرور موجود باشد. `long-polling` ثبت می شود؛ در اجرای لوکال به جای آن یک رشته `/hook/{bot_id}/{secret}` روشن می شود.
 3. هر پیام ورودی با نسخه «زنده» اجرا می شود، داده زیر قفل هر بات ذخیره و پاسخ ها با کیبورد پاسخ. ارسال می شوند (Reply Keyboard).
 4. انتشار نسخه جدید فقط اشاره گر نسخه زنده را جابه جا می کند؛ داده های واقعی دست نمی خورند و. بازگشت به نسخه قبل هم همین طور است.
 5. با راه اندازی دوباره سرور، همه بات های زنده دوباره وصل می شوند.
- وضعیت آزمون: انتشار روی تلگرام از نسخه آنلاین آزموده شده و کار می کند. انتشار و پرداخت روی بله از هم آزموده شده است Render سرور.
- پلتفرم اصلی دمای دآوری **تلگرام** است؛ پرداخت ریالی روی بله اجرا می شود.

پرداخت آنلاین

ثبت نام یا سفارش پولی فقط وقتی قطعی می شود که خود پلتفرم (بله یا تلگرام) پرداخت موفق را اطلاع فعال می شود و ایجنت برای کارگاه های پولی درباره Spec در `require_payment` دهد؛ این رفتار با گزینه آن می پرسد.

1. با وضعیت «در (پیش فرض ۱۵ دقیقه) `payment_hold_minutes` پس از تأیید اطلاعات، جا به مدت. ارسال می شود `sendInvoice` انتظار پرداخت» نگه داشته و صورت حساب با.
2. بررسی می کند که صورت حساب مال همین کاربر است و `pre_checkout_query`، پیش از کسر وجه. مهلتش تمام نشده.
3. ثبت نام قطعی، کد پیگیری صادر و شماره تراکنش ذخیره می شود `successful_payment` با دریافت.

یک رشته پس‌زمینه هر ۶۰ ثانیه رزروهای منقضی را آزاد و نفر بعدی فهرست انتظار را با صورت حساب 4. جدید خبر می‌کند؛ پرداخت دیر هنگام رد می‌شود.

آزمون	توکن پرداخت	ارز	پلتفرم
توکن WALLET-TEST-1111111111111111 و پرداخت واقعی با کارت تیم (انجام شده)	بله @botfather از	ریال (تومان $\times 10$)، واریز به کیف پول بله	بله
Stripe در حالت Test	بخش BotFather، از Payments	دلار با نرخ تبدیل قابل تنظیم (درگاه‌های تلگرام ریال ندارند)	تلگرام
دکمه «پرداخت آزمایشی» با همان مسیر تأیید	لازم نیست	Spec همان ارز	شبیه‌ساز

آزمون‌های خودکار این موارد را می‌سنجند: بدون پرداخت کد پیگیری صادر نمی‌شود، پس از مهلت جا آزاد می‌شود، و جای آزاد شده با صورت حساب به فهرست انتظار می‌رسد.

منابع داده و مدل داده

تنها منبع ورودی محصول، توضیحات صاحب کسب‌وکار در گفتگو و پیام‌های مشتریان بات است؛ از داده در SQLite در تولید و Postgres (Neon) استفاده نمی‌شود. ذخیره‌سازی روی RAG بیرونی، خزش وب یا اجرای لوکال است.

جدول	محتوا
users	(bcrypt) ایمیل، نام، هش رمز
bots	نسخه زنده، webhook مالک، قالب، پلتفرم، توکن، راز وضعیت ایجنت، خلاصه نیازمندی‌ها (حافظه ایجنت)
bot_versions	آزمون‌ها، گزارش آزمون، یادداشت، Spec، شماره نسخه تغییر، فرض‌های ایجنت
messages	(step) گفتگوی صاحب کسب‌وکار و ایجنت + ردیف‌های گام برای نمایش زنده کار ایجنت
bot_data	live جدا برای JSON، حالت زمان اجرای هر بات به صورت sandbox و

برای کارگاه: ثبت نام‌ها و فهرست انتظار به تفکیک شناسه جلسه؛ برای سفارش: `bot_data` محتوای فهرست سفارش‌ها با اقلام، مبالغ و اطلاعات مشتری؛ و برای هر دو، وضعیت گفتگو و سبب هر کاربر. صاحب کسب و کار این داده‌ها را در تب «داده‌ها» می‌بیند.

و امنیت API

نیاز دارند و فقط بات‌های همان کاربر JWT به توکن webhook همه مسیرها به جز ثبت نام، ورود، سلامت و را برمی‌گردانند.

مسیر	کار
<code>POST /api/auth/register , /login</code>	با اعتبار ۱۴ JWT ساخت حساب و ورود، برگرداندن روز
<code>GET/POST /api/bots</code>	فهرست بات‌ها، ساخت بات با پیام اول
<code>GET/POST /api/bots/{id}/messages</code>	گفتگو با ایجنت؛ ایجنت در پس زمینه اجرا و رابط هر ۱.۲ ثانیه بررسی می‌کند
<code>GET /api/bots/{id}/versions/{vid}</code>	آزمون‌ها و گزارش یک نسخه، Spec
<code>POST /api/bots/{id}/sim , /sim/reset</code>	شبیه ساز چند کاربره
<code>POST /api/bots/{id}/publish , /unpublish</code>	انتشار یک نسخه روی بله یا تلگرام و توقف
<code>GET /api/bots/{id}/data</code>	ثبت نام‌ها و سفارش‌های واقعی یا آزمایشی
<code>POST /hook/{bot_id}/{secret}</code>	دریافت پیام‌های پلتفرم

اقدامات امنیتی

- پذیرفته می‌شود Pydantic ایجنت کد تولید نمی‌کند؛ خروجی مدل فقط پس از اعتبارسنجی.
- فقط به صورت ماسک شده برمی‌گردد API هش می‌شوند؛ توکن بات در پاسخ‌های bcrypt رمزها با.
- یک راز تصادفی ۱۲۸ بیتی دارد که با مقایسه زمان ثابت بررسی می‌شود webhook هر.
- محافظت شده است path traversal سرو فایل‌های فرانت در برابر.
- هستند و در مخزن کد نیستند Render کلیدها فقط در متغیرهای محیطی.

راه‌اندازی و دیپلوی

می‌سازد و مرحله دوم آن را کنار Node 20 است: مرحله اول فرانت را با Docker کل محصول یک کانترینر اجرا می‌شود؛ کارهای ایجنت، رشته‌های worker اجرا می‌کند. عمده‌ایک Python 3.11 روی FastAPI و قفل هر بات درون همان فرآیندند polling.

اجرای لوکال

```
cd backend
python -m venv .venv
.venv/Scripts/pip install -r requirements.txt
cp .env.example .env
.venv/Scripts/uvicorn app.main:app --port 8000

cd ../frontend
npm install
npm run dev
```

ثبت‌نام، سؤال، ساخت، LLM: `python tests/test_flow.py` آزمون سرتاسری بدون مصرف سهمیه (اصلاح، شبیه‌ساز، تغییر و رگرسیون با مدل ساختگی).

دیپلوی رایگان

- آن برداشته می‌شود Connection string ساخته و Neon رایگان در Postgres دیتابیس.
- منطقه Docker برابر Runtime با GitHub از مخزن عمومی Web Service یک Render در. ساخته می‌شود Free و پلن Frankfurt.
- آدرس عمومی. `DATABASE_URL`، `LLM_API_KEY`، `LLM_MODEL`، `JWT_SECRET`: متغیرهای محیطی. استفاده می‌شود webhook خودکار خوانده و برای `RENDER_EXTERNAL_URL` سرویس از `/api/health` health check برای جداول دیتابیس در اولین اجرا خودکار ساخته می‌شوند. مسیر.
- استفاده می‌شود.

هزینه‌ها

صفر است؛ همه سرویس‌ها روی پلن رایگان‌اند. ارقام مقیاس تقریبی‌اند و باید با MVP هزینه فعلی اجرای قیمت‌های روز سرویس‌دهنده‌ها بازبینی شوند.

بخش	اکنون (MVP)	پس از رشد (تقریبی)
سرور (Render)	رایگان، با خواب پس از ۱۵ دقیقه	پلن پولی بدون خواب، حدود ۷ دلار در ماه

بخش	(MVP) اکنون	پس از رشد (تقریبی)
دیتابیس (Neon)	رایگان	پلن پولی در صورت نیاز به ظرفیت بیشتر
مدل زبانی (Gemini)	سطح رایگان، با سقف درخواست روزانه	پرداخت به ازای توکن
اجرای بات‌ها	LLM بدون هزینه	LLM بدون هزینه

نکته اقتصادی مهم: **مدل زبانی فقط در زمان ساخت یا تغییر بات مصرف می‌شود**، نه در گفتگوی ندارد. هر ساخت یا تغییر حدود ۴ تا ۶ LLM مشتریان با بات، یک بات با هزاران پیام مشتری هیچ هزینه و قرارداد موتور است Schema فراخوانی مدل است؛ بیشتر حجم هر فراخوانی از

محدودیت‌های فنی و نقشه راه

محدودیت	اثر	راه حل بعدی
فقط دو قالب (کارگاه، سفارش)	درخواست خارج از این دو با توضیح رد می‌شود	قالب‌های نوبت‌دهی، پشتیبانی و نظرسنجی
بدون بازگشت خودکار وجه و یادآوری زمان‌دار	بازگشت وجه را کسب و کار دستی انجام می‌دهد	بازگشت وجه از کیف پول و زمان‌بند پیام
سقف درخواست سطح رایگان Gemini (خطای 429 و 503 در) (آزمون‌ها دیده شد)	ساخت بات ممکن است کند شود یا ناموفق بماند	چرخش خودکار بین مدل‌ها (پیاپی شده)، سرویس‌دهنده دوم مثل پلن پولی، Groq
خواب سرور رایگان پس از ۱۵ دقیقه	اولین بازدید ۳۰ تا ۶۰ ثانیه طول می‌کشد	پلن پولی یا میزبان دائم
یک فرآیند و قفل درون حافظه	مقیاس افقی ممکن نیست	صف کار و قفل در دیتابیس
درگاه‌های تلگرام ریال ندارند	پرداخت تلگرام فقط برای آزمون مناسب است	بله پلتفرم اصلی پرداخت ریالی
بدون بازیابی رمز و نقش‌های تیمی	هر حساب یک کاربر دارد	MVP خارج از دامنه